



HOLON x KBI AI AGENTS HACKATHON 2025

Overview

Welcome to the HOLON x KBI AI Agents Hackathon 2025, a two-week virtual sprint for the best university minds to design and build real-world AI agents. Hosted by [Holon AI](#) and [Königsberger Bridges Institute \(KBI\)](#), this hackathon combines practical AI challenges with real deployment, mentorship, and an opportunity to work in Hong Kong/China on real AI use cases.

Organizers:

Holon AI is an AI company building outcome-driven agentic tools for small and medium-sized businesses (SMBs) across Asia. Our mission is to simplify business operations by creating AI agents that produce measurable results and not just predictions or chats. Holon develops systems for sales automation, marketing performance, analytics, accounting, and internal productivity. Our technology stack includes open-source LLMs, agent frameworks like LangChain, and secure cloud infrastructure hosted in China-compliant environments. We partner with local businesses and startups to deploy these tools directly into operations, measuring success by ROI and adoption rather than vanity metrics.

KBI is an EU-based nonprofit running the International Data Science Olympiad (IDSOL) and the International Blockchain Olympiad (IBCOL). It aims to elevate global innovation by training students and innovators through immersive competitions.



Timeline

- Hackathon Launch: April 10, 2025
- Submissions Due: April 25, 2025



Who Can Join

- Open to all students (B.tech/M.tech/IDD)
- Teams of 1–2 only
- Basic Python + Web knowledge recommended



Prizes

- Top 2 teams win internships with Holon in Hong Kong/China
- All travel and stay expenses covered (*Round-trip flight tickets, Comfortable accommodation, Daily food expenses*)

Tracks

You can choose **one** of the following challenges



Track 1: Multilingual Note-Taking Agent



Problem

Office workers in Hong Kong often attend fast-paced, multilingual meetings (English, Mandarin, Cantonese). Manually taking notes is inefficient and stressful, especially for less tech-savvy users.



Goal

Build a multilingual note-taking AI agent that:

- Joins or records meetings (online or in-person)
- Transcribes and summarizes spoken content
- Allows users to search, view, and share notes
- Supports English, Mandarin, and Cantonese



MVP Scope

Inputs:

- Upload recorded audio or simulate meeting recordings (Zoom/Google Meet or offline audio is fine)

Agent Capabilities:

- Transcribe audio using Alibaba Cloud ASR or any local open-source ASR
- Summarize meeting notes using LangChain + Qwen/DeepSeek or any other LLM (avoid openai)
- Search within transcriptions (by keyword or topic)
- Export notes as PDF or shareable text

Outputs:

- Concise meeting summary with action points
- Keyword-searchable transcript
- Exportable report (PDF)

⚙️ Tech Stack Overview (Flexible)

Step	Tech Stack	Role	Purpose	FE/BE
1	React	User Interface	For uploading audio, viewing summaries, and downloading PDFs	Frontend
2	FastAPI	API Server	Bridges frontend and backend, handles requests	Frontend ↔ Backend
3	Whisper or Alibaba ASR	Speech Recognition	Converts meeting audio to text, supports multilingual input	Backend
4	LangChain + Qwen/DeepSeek	Summarization & Action Extraction	Extracts key points, decisions, and action items	Backend
5	SQLite	Search Indexing & Storage	Lightweight DB for searchable transcripts	Backend
6	PDF Generator (e.g., fpdf)	Document Output	Generates shareable PDF reports from summaries	Backend
7	Local Deployment/ Alibaba Cloud	Deployment Option	Supports local testing, privacy, or China-specific hosting	Deployment Env
8	Local Deployment	Testing Environment	Enables local testing without real Meta API	Deployment

🧠 Agentic Nature

This is not just a voice-to-text tool. Your AI should:

- Understand key points
- Structure summaries logically
- Recognize action items and decisions
- Adapt to multilingual content

Evaluation Criteria

Criteria	Weight
Meeting Summary Clarity	30%
Multilingual Transcription	25%
Search & Sharing Functions	20%
Ease of Use	15%
Agentic Reasoning	10%

Deliverables

- Public GitHub repo
- Working demo (local or recorded walkthrough)
- Sample input/output files
- 1-pager summary of agent design + screenshots

Tips

- Start with offline audio files to reduce complexity
- Let the agent infer who said what (e.g., via tone or pause)
- Use simple UI for non-technical users (big buttons, clear flow)

Submission Checklist

- GitHub repo with README
- Demo video or deployed link
- Meeting summary output (PDF)
- Transcription + search demo screenshots
- Brief PDF with design logic

 Focus on clarity, structure, and the multilingual edge — not flashy UI. Let your agent **listen, think, and write smart notes.**

Track 2: Performance Marketing Agent



Problem

Small businesses often waste advertising budgets on Meta (Facebook/Instagram) due to poor targeting, inconsistent creatives, and lack of continuous optimization. Marketing teams don't have time to test and adapt ads at scale.



Goal

Build an AI agent that:

- Sets up Meta Ads campaigns (simulated or via mock API)
- Allocates budgets and optimizes targeting
- Rotates creatives and improves underperforming ones
- Summarizes campaign performance clearly



MVP Scope

Inputs:

- Choose sample creative assets (mock Canva outputs)
- Simulate Meta Ads setup via local mock API or JSON schema

Agent Capabilities:

- Define campaign objectives, targeting, copy, and budget
- Rotate ad copy and visuals to improve performance
- Track performance KPIs and optimize live campaigns
- Generate weekly campaign reports with data-driven suggestions

Outputs:

- Campaign setup summary (structure, copy, audience)
- Visual performance dashboard (CTR, CPC, spend)
- Agent-generated optimization steps

Agentic Nature

The agent should:

- Set up multiple ad sets based on goals
- React to data and change copy/visuals accordingly
- Recommend daily actions (e.g., pause poor ad set)

Tech Stack Overview (Flexible)

Step	Tech Stack	Role	Why It's Used	FE/BE
1	React	Dashboard UI	For setting campaigns, viewing performance, and showing optimization tips	Frontend
2	FastAPI	API Server	Connects frontend to backend and handles campaign logic	Backend
3	LangChain + Qwen/Deep Seek	Reasoning & Copy Generation	Creates ad copy, targeting, and improves based on data	Backend
4	Canva Asset Folder	Mock Creative Assets	Holds sample visuals and ad copy to simulate real campaigns	Backend (Static)
5	JSON Structure	Meta API Simulation	Represents the campaign setup logic in a realistic way	Backend
6	CTR/CPC Simulator	Performance Simulation	Mimics ad performance to test optimization logic	Backend
7	Python	Core Logic Language	Used to write backend logic and simulations	Backend
8	Local Deployment	Testing Environment	Enables local testing without real Meta API	Deployment

Evaluation Criteria

Criteria	Weight
Functionality & Agent Logic	30%
UX Simplicity	20%
Optimization Flow	20%
Realism of Ad Simulation	20%
Reporting & Summary	10%

Deliverables

- Public GitHub repo
- Campaign setup flow demo
- Agent-generated optimization summary
- Visual dashboard screenshots
- PDF walkthrough of logic and prompt design

Tips

- Simulate Meta Ads logic in code instead of using real API
- Include examples where ad set A gets paused due to poor CTR
- Use mock visuals but realistic campaign flow

 Build an AI that actually acts like a **performance marketer**: sets, monitors, learns, and adapts.

Track 3: Accounting Reconciliation & Advisory Agent



Problem

CPA firms and finance teams waste hours manually matching transactions across systems. Existing tools fail when there are partial matches or mismatched metadata. They also lack feedback and improvement over time.



Goal

Build an AI agent that:

- Matches bank CSV to ledger CSV
- Escalates unclear cases for human input
- Learns from feedback and improves
- Summarizes the month's financial activity in plain language



MVP Scope

Inputs:

- [bank.csv](#) and [ledger.csv](#) from mock data

Agent Capabilities:

- Perform fuzzy matching using logic and heuristics
- Highlight discrepancies and confidence scores
- Accept feedback and update matching patterns
- Summarize key financial patterns for a monthly review

Outputs:

- Reconciliation table with match status
- Unmatched entries and reasoning
- Executive summary report (text + table)

Agentic Nature

Agent must:

- Learn and update logic from user feedback
- Decide when to escalate based on confidence threshold
- Generate human-style financial summaries

Tech Stack Overview (Flexible)

Step	Tech Stack	Role	Why It's Used	FE/BE
1	React	User Interface	For uploading CSVs, displaying matched results, and showing summaries	Frontend
2	FastAPI	API Server	Connects frontend to backend and processes reconciliation logic	Backend
3	LangChain + Qwen/DeepSeek	Financial Summary Generator	Creates plain-language financial summaries based on reconciled data	Backend
4	Pandas / DuckDB	CSV Matching Engine	Performs fuzzy matching and reconciliation logic on bank and ledger files	Backend
5	Local CSV Storage	File Storage	Simplifies input/output by using local files instead of a database	Backend (File I/O)

Evaluation Criteria

Criteria	Weight
Matching Logic + Accuracy	30%
Feedback Learning Loop	25%
Usability	20%
Agentic Reasoning	15%
Reporting Quality	10%

Deliverables

- Public GitHub repo
- Demo of reconciliation flow
- Match result tables and heatmaps
- Financial summary sample output
- PDF explaining design logic

 Think like a digital junior accountant. Match, escalate, learn — and advise.



Track 4: AI Business Intelligence Agent : Smart Dashboards for Decision-Making



Problem

SMEs in Asia rely on fragmented, hard-to-use tools to make decisions. Business owners and managers often struggle to get timely answers from platforms like Google Analytics or internal systems. Dashboards exist but don't speak the user's language literally or contextually.



Goal

Build a multilingual BI(Business Intelligence) agent that:

- Connects to **Google Analytics** (mock or real)
- Accepts queries in chatbot format
- Uses predefined + evolving question sets to answer business queries
- Surfaces actionable insights from connected agents (ads, accounting, etc.)
- Returns responses in English, Mandarin, or Cantonese



MVP Scope

Inputs:

- Google Analytics (mock data or API output in JSON)
<https://support.google.com/analytics/answer/7586738?hl=en>
- Optional: outputs from other agents (ads optimizer, accounting summaries)

Agent Capabilities:

- Accept user questions via a simple chat interface
- Match user questions to a predefined library (e.g., "What was my bounce rate last month?")
- Expand/improvise these queries using LLM-based NLP
- Return data-driven insights with supporting charts or text summaries

Outputs:

- Chat-like interface for business queries
- Data visualization + natural language summaries
- Multilingual response support

Agentic Nature

Your BI agent should:

- Select the best dataset + question template based on intent
- Offer follow-up questions for deeper insight
- Learn which types of questions are being asked and evolve its suggestions
- Speak to non-technical users ("Your top page dropped 30% this week")

Tech Stack Overview (Flexible)

Step	Tech Stack	Role	Why It's Used	FE/BE
1	React	Chat UI	Enables users to ask business questions through a conversational interface	Frontend
2	FastAPI	API Server	Receives queries, runs agent logic, and returns data-driven answers	Backend
3	LangChain + Qwen/DeepSeek	LLM Reasoning Engine	Understands user intent, expands queries, and generates natural-language responses	Backend
4	Google Analytics Mock Data (CSV/JSON)	Data Source	Provides web performance data (visits, bounce rate, etc.) for analysis	Backend
5	Pandas / DuckDB	Data Parser & Query Engine	Parses GA data and extracts relevant metrics	Backend
6	SQLite	Query Template Mapping	Maps user queries to stored question templates or query patterns	Backend
7	Plotly / ECharts	Visualization Engine	Displays insights in charts/graphs along with textual summaries	Backend

Evaluation Criteria

Criteria	Weight
Business Relevance of Questions	30%
Insight Accuracy	25%
Agent Flow & Query Matching	20%
Multilingual Clarity	15%
User Interface	10%

Deliverables

- Public GitHub repo
- Chatbot UI demo (recorded or live)
- Screenshot of at least 5 predefined query+response flows
- 1 auto-generated multilingual summary report
- 1-pager PDF on prompt mapping + architecture

 Don't just build a dashboard — build a **multilingual data advisor** that thinks, recommends, and explains.

Getting Started

This section will help you quickly get started with the hackathon.

Recommended Tools

- Python 3.10+
- VSCode or Jupyter Notebook
- GitHub account (public repo submission required)
- React + FastAPI for quick UI
- LangChain framework (latest stable)

Suggested Workflow

1. **Choose a Track** – Pick from the 4 outlined tracks.
2. **Review MVP Goals** – Start with the core features listed.
3. **Use Mock Data** – You're encouraged to simulate APIs or use CSV/JSON.
4. **Build Agent Logic First** – Focus on decisions, summaries, flows.
5. **Add UI Later** – Once logic works, build a simple frontend.
6. **Record a Demo** – Showcase your working features clearly.

Cloud Deployment

- Deployment to Alibaba Cloud is optional for submission.
- Finalists will get access to Holon's Alibaba Cloud environment.
- Please ensure your code is Docker-compatible or deployable locally.

 **Tip:** If you're unsure about cloud, just **focus on getting it working locally**. We'll help finalists deploy it properly post-hackathon.

 For help and guidance, join the Discord: <https://discord.gg/J8uw6mWBqF>

 For questions: hackathon@holonai.ai , atul@holonai.ai (+852 6994 1597)